Stage One Follow-up Report: Checks Through E1, Code Reviews and Proposed Corrections

Oct 14, 2026 · @Shantha

Status at 11:35 UTC, 15 October: the approved read-only run is **complete through E1**: 21 statements on localhost between 11:03 and 11:13 UTC, one at a time, every checksum and counter check passed, no stop rule triggered. **R20.2r was not run**, the original R20.2 was not rerun and G1 was not run. The LKR 298,375.00 difference is one voucher-payment journal posted twice for voucher 7134 (section 3). E1′s plan meets conditions 2–5; condition 1 turns on whether two MySQL Notes count as warnings (section 4). The evidence folder is complete with SHA256SUMS (section 8). **Nothing was installed and no data, schema, permission or staff assignment was changed.** Sections 1–5 and 8 are new; sections 6, 7 and 9 are unchanged from 14 October.

Kind	Item	Status
Completed check	Code reviews F-1 to F-4, F-6, F-7 (section 6)	done, read-only; the 23 reviewed files equal the target′s by SHA-256 (checked 23:20 UTC)
Completed check	F-5: five files against R0′s manifest	done: all five match. The whole tree also still matches R0′s manifest (23:19 UTC)
Completed check	Statement texts against the Revision 1 source	all 12 found in the Revision 1 doc, every SHA-256 matches
Completed check	Target verification and T1 to E1 (sections 1–5)	done, read-only, 15 Oct 11:03–11:13 UTC after your sign-in; T1 = 8.4.7 / 3306 / avenir / root@localhost; 21 statements, all checksums matched, all eight counter checks passed, no stop rule triggered; R20.2r not run
Completed check	Evidence folder (section 8)	done: C:\wamp64\www\avenir\writable\AvenirEvidence\2026-10-14-stage1 (your corrected location), 114 files, SHA256SUMS verified on your computer with 0 mismatches; reports\ still empty (PDF exports pending)

Kind	Item	Status
Completed check	R20.2 recovery	only partly recoverable; its result was not recovered: the browser log still held the response at 11:20 UTC on 15 Oct, but only its first part (phpMyAdmin's settings form, with a session token) can be read; nothing saved, pending your decision
Proposed correction	Stage A guards, Stage B rules, Stage C data items (section 7)	proposal only
Installed change	—	**none**

1. Target verification: localhost MySQL and schema avenir

Sign-in. You signed in to phpMyAdmin 5.2.3 yourself. At 10:54:06 UTC on 15 October the sign-in form was gone and the page showed server 1 (MySQL), database avenir, account root. I did not see or touch the form. **[Observed]**

One tab only. Before the first statement, the two other browser-pane tabs were closed (11:00 UTC): a second phpMyAdmin tab that was showing its own sign-in form, and the local application's intake page, which was signed out (its polling calls were returning 401). The pane's network log shows the 21 statements of this run as one unbroken series of SQL requests (ids 12397–12417) with nothing between them. **[Observed]**

T1 (sent 11:03:42.531 UTC, 368 ms, HTTP 200, 1 row): mysql_version **8.4.7** · port **3306** · selected_schema **avenir** · signed_in_account **root@localhost**. This is the Stage One target. **[Observed]**

H0 (11:04:07 UTC, 1 row): global_max_execution_time **0** · session_max_execution_time **0** · performance_schema_on **1** · mysql_version 8.4.7. Both timeout settings are at MySQL's default, so the H1 counters can show whether each hint was applied, and performance_schema is on, so H1 has something to read. Neither stop rule was triggered. **[Observed]**

Not verified: that the local application was idle. The pane's own application tab was signed out; whether anyone else used the local application between 11:03 and 11:13 UTC is not established. The timeout counters moved only by our own statements (section 2), which is consistent with no other timed SELECT running. **[Not established]**

These are localhost readings. They say nothing about the live staff system, which was not accessed.

2. Run log: each statement, checksum, capture, timeout-counter check

What was done. 21 statement executions, 11:03:42 to 11:12:33 UTC on 15 October, strictly one at a time; each response was saved before the next statement was sent. Steps 22–23 (R20.2r and its H1) were **not run**, by your instruction. G1 (the uncapped reference) was not run. The original R20.2 was not rerun. **[Observed]**

Gate, for every statement. The SHA-256 of the text about to be sent was computed in the page and compared with the Revision 1 value; the same 12 texts had been re-verified against the Revision 1 document in the session workspace. All 21 matched. The sender accepts one statement only, SELECT, or EXPLAIN for E1 alone (EXPLAIN ANALYZE refused), and refuses while another statement is in flight or when a sign-in form is present. It sends through phpMyAdmin's own SQL handler with "show all rows".

Capture, for every statement. The raw JSON response was kept before any parsing; phpMyAdmin's session token values were replaced by "[token removed]" and nothing else changed; its SHA-256 was computed in the browser, verified again after transfer to the session workspace, and once more on your computer (SHA256SUMS). The result table was parsed into a pipe-separated rows file and its count compared with phpMyAdmin's "Showing rows … (N total)". For G1c phpMyAdmin printed "executed successfully" without a total; its own export link carried unlim_num_rows=1, which equals the parsed count. E1 (EXPLAIN) prints no total; its single plan row is the whole result.

Step	Statement	Sent (UTC)	Round trip	phpMyAdmin "Query took"	Rows (parsed = total)	HTTP	Counter after
1	T1	11:03:42.531	368 ms	0.0002 s	1	200	—
2	H0	11:04:07.924	271 ms	0.0013 s	1	200	—
3	H1 (baseline)	11:04:34.270	425 ms	0.0470 s	3	200	set 0 · failed 0 · exceeded 0
4	M1	11:04:52.832	906 ms	0.0086 s	30	200	
5	H1	11:05:15.074	617 ms	0.0006 s	3	200	set 1 · 0 · 0 (+1)
6	M2	11:05:35.386	809 ms	0.0633 s	18	200	
7	H1	11:05:55.107	5,478 ms	0.0028 s	3	200	set 2 · 0 · 0 (+1)
8	G1c	11:06:21.497	463 ms	0.0067 s	1	200	

Step	Statement	Sent (UTC)	Round trip	phpMyAdmin “Query took”	Rows (parsed = total)	HTTP	Counter after
9	H1	11:07:27.196	473 ms	0.0056 s	3	200	set 3 · 0 · 0 (+1)
10	G2r	11:07:46.834	2,438 ms	0.3789 s	38	200	
11	H1	11:08:12.131	520 ms	0.0039 s	3	200	set 4 · 0 · 0 (+1)
12	G3	11:08:32.820	1,853 ms	0.0097 s	78	200	
13	H1	11:09:01.064	412 ms	0.0008 s	3	200	set 5 · 0 · 0 (+1)
14	G4	11:09:21.347	877 ms	0.2751 s	38	200	
15	H1	11:09:59.759	417 ms	0.0008 s	3	200	set 6 · 0 · 0 (+1)
16	L1	11:10:19.508	1,839 ms	0.7020 s	11	200	
17	H1	11:10:47.573	349 ms	0.0064 s	3	200	set 7 · 0 · 0 (+1)
18	L2	11:11:07.881	855 ms	0.1406 s	41	200	
19	H1	11:11:27.971	495 ms	0.0059 s	3	200	set 8 · 0 · 0 (+1)
20	E1	11:11:50.053	350 ms	0.0354 s	1 (plan)	200	
21	H1 (new baseline)	11:12:32.607	444 ms	0.0008 s	3	200	set 9 · 0 · 0

Timeout checks (section 3 of Revision 1). After each of the eight capped statements, Max_execution_time_set rose by exactly 1, Max_execution_time_set_failed stayed 0 and Max_execution_time_exceeded stayed 0. MySQL therefore set a 60-second timeout for every capped SELECT, and none ran over. The step-7 round trip of 5.5 s was network or phpMyAdmin time; the server reported 0.0028 s. E1′s own H1 reading (step 21) moved “set” from 8 to 9: the EXPLAIN of a hinted SELECT also counts as a set timeout. Revision 1 left this unestablished and did not use it as a test; it is recorded here as the new baseline. **[Observed]**

Stop rules. None was triggered: no error, no warning, no HTTP status other than 200, no missing table, no row-count mismatch, no checksum difference, no sign-in prompt, no second statement in flight; T1 and H0 as expected; every counter check passed; G2r′s amount total equals 6,535,170.97 (section 3). The two Notes that E1 returned are discussed in section 4.

Statement times. The longest server time was L1 at 0.702 s; the eight capped statements together took under 1.5 s of server time. The 60-second caps were never approached.

The raw responses, rows files, per-step metadata and RUN_LOG.txt are in followup-run/ of the evidence folder (section 8).

3. The LKR 298,375.00 difference: findings and uncertainty

The finding in one sentence. The whole difference is one voucher-payment journal posted twice for voucher **7134**: the ledger holds two identical journals under that number, each crediting 1-6-8-9750 with 298,375.00, where every other sup = 0 voucher has exactly one. **[Observed]**

What the four reads show [Observed]

G1c (1 row). Of the 38 sup = 0 authorised vouchers, exactly one has credits to 1-6-8-9750 that differ from its amount: voucher 7134, amount 298,375.00, credits 596,750.00 from 2 journals. The excess, 298,375.00, is the R0-minus-R23.1 difference (6,833,545.97 − 6,535,170.97).

G2r (38 rows, one per voucher number). Built-in checks: SUM(amount_sup0_authorised_rows) = **6,535,170.97**, equal to R23.1, so the run continued; SUM(cr_9750) = **6,833,545.97**, which reproduces R0's figure. For all 38 numbers: one voucher row (no shared numbers), one line in v_inv whose amount equals the voucher amount, no rows under the number in account_runing_2024, no debit to 1-6-8-9750 (dr_9750 = 0), every credit to 1-6-8-9750 is a "VOUCHER PAYMENT" row, and none carries Invoice_Num '0'. 37 numbers have cr_9750 equal to the voucher amount; 7134 alone has twice it. Shared numbers cannot occur any more: M2 shows a unique index on voucher.Vnum (uq_tpd_voucher_vnum).

G3 (78 rows = 39 journals × 2 legs). Every journal has exactly two legs and balances; every row is a "voucher payment" row with aflag 3 and no Bank_Date. 37 vouchers have one journal. Voucher 7134 has two:

Journal	Entered at (Dt)	Entering login id	Debit	Credit	Date	Invoice_Num
26061	2026-07-13 15:24:03	75	6-9-23-11013 298,375.00	1-6-8-9750 298,375.00	2026-07-10	the line's Invoice
26075	2026-07-13 16:28:48	1	6-9-23-11013 298,375.00	1-6-8-9750 298,375.00	2026-07-10	the line's Invoice

The two journals are identical apart from their id, entry time (64 minutes 45 seconds apart) and entering login. Both carry the line's Invoice in Invoice_Num, not '0'.

G4 (38 rows). Each of the 38 vouchers has one line, RCType 1. Voucher 7134's line (id 7204) is to account 6-9-23-11013 for 298,375.00, dated 2026-07-10. Its amount equals the voucher amount, so the voucher itself is not doubled; only its ledger posting is.

What follows [Deduced]

The expense account 6-9-23-11013 is debited twice and the payable 1-6-8-9750 credited twice for one voucher. Until the accountant corrects it, the ledger overstates both by LKR 298,375.00. The voucher was finalized with bank date 2026-07-13 (Stage One R23.1); the bank-side posting is outside these reads (below).

The posting path is consistent with the per-line route V1 (section 6: VoucherPaymentController::update, which copies the line's Invoice into Invoice_Num and has no Aflag guard) having run twice for the same voucher, under two different logins. The V4 route writes Invoice_Num '0' and posts k times at once; neither mark is present. This is consistency with the code, not proof of the sequence of events.

What is not established [Not established]

Why it was posted twice. No audit covers 13 July 2026 (tpd_audit starts on 11 October), and the legacy screens record no edits or deletions.

Which journal is the duplicate. The two are identical in amounts and accounts. Reversing one is an accounting decision; I propose no correcting journal (section 7, Stage C).

Whether the bank leg was also posted twice. No journal under number 7134 debits 1-6-8-9750 (dr_9750 = 0 for all 38 numbers), so the 9750-to-bank clearing for these vouchers is not recorded under their voucher numbers, and G3 cannot show it. Whether the Commercial Bank account 1-6-8-10380 was credited once or twice for this voucher needs a separate read of the bank ledger, which was not in the approved set.

Scope. These are localhost readings on 15 October. They do not describe the live staff system.

Data note. The v_inv Invoice column (line_invoice in G3 and G4) holds free text typed by staff: dates, reference numbers and in some rows names and numbers that may be account numbers. The evidence files keep what the database returned; this report does not quote those values.

4. E1: the captured plan and each required condition

E1 was sent at 11:11:50.053 UTC as EXPLAIN FORMAT=TREE, without ANALYZE, with the approved text (SHA-256 3aae853b…). It returned in 350 ms with HTTP 200, one plan row and no error. The plan was saved as followup-run/20_E1_plan.txt (SHA-256 320b8c96…). **[Observed]**

The plan, as returned:

```
-> Sort: voucher_month
    -> Table scan on <temporary>
        -> Aggregate using temporary table
            -> Nested loop left join  (cost=16424 rows=39078)
                -> Nested loop left join  (cost=12317 rows=39078)
                    -> Nested loop antijoin  (cost=8210 rows=39078)
                        -> Nested loop antijoin  (cost=2906 rows=5583)
                            -> Filter: (v.Aflag = 1)  (cost=952 rows=798)
                                -> Table scan on v  (cost=952 rows=7975)
                            -> Filter: (upper(a.Discription) like 'VOUCHER
PAYMENT%')  (cost=12.3 rows=7)
                                -> Index lookup on a using Recpt_No
(Recpt_No=v.Vnum)  (cost=12.3 rows=7)
                        -> Filter: (upper(b.Discription) like 'VOUCHER
PAYMENT%')  (cost=12.3 rows=7)
                            -> Index lookup on b using Recpt_No
(Recpt_No=v.Vnum)  (cost=12.3 rows=7)
                    -> Filter: ((cur.d = cast(v.W_Date as date)) and (cur.amt
= v.Amount))  (cost=0.25..0.25 rows=1)
                        -> Single-row covering index lookup on cur using
<auto_distinct_key> (d=cast(v.W_Date as date), amt=v.Amount)
(cost=4852..4852 rows=1)
                            -> Union materialize with deduplication
(cost=4852..4852 rows=22728)
                                -> Filter: (upper(a.Discription) like
'VOUCHER PAYMENT%')  (cost=1289 rows=11364)
                                    -> Table scan on a  (cost=1289
rows=11364)
                                -> Filter: (upper(a.Discription) like
'VOUCHER PAYMENT%')  (cost=1289 rows=11364)
                                    -> Table scan on a  (cost=1289
rows=11364)
                -> Filter: ((`old`.d = cast(v.W_Date as date)) and (`old`.amt
= v.Amount))  (cost=0.25..0.25 rows=1)
                    -> Single-row covering index lookup on old using
<auto_distinct_key> (d=cast(v.W_Date as date), amt=v.Amount)
(cost=6735..6735 rows=1)
                        -> Union materialize with deduplication
(cost=6735..6735 rows=31800)
                            -> Filter: (upper(b.Discription) like 'VOUCHER
PAYMENT%')  (cost=1777 rows=15900)
                                -> Table scan on b  (cost=1777 rows=15900)
                            -> Filter: (upper(b.Discription) like 'VOUCHER
PAYMENT%')  (cost=1777 rows=15900)
                                -> Table scan on b  (cost=1777 rows=15900)
```

phpMyAdmin also showed two Notes: “Note: #1276 Field or reference 'avenir.v.Vnum' of SELECT #8 was resolved in SELECT #1” and the same for SELECT #9. These are MySQL level-Note diagnostics saying that the two NOT EXISTS subqueries take v.Vnum from the outer query, which is what a correlated NOT EXISTS does. MySQL's three diagnostic levels are Note, Warning and Error; these are Notes. **[Observed]**

The conditions of Revision 1, section 2

Condition	Reading of the plan	Result
1. A plan with no error and no warning	No error. No Warning-level diagnostic. Two Note-level diagnostics (#1276) as above.	**Passes on MySQL's levels; your call** whether a Note counts as a warning for this rule
2. Each UNION set appears once, as a union materialisation, outside any per-voucher loop; not “invalidate on row”; not inside a dependent subquery	“Union materialize with deduplication” appears once for cur (rows=22728) and once for old (rows=31800), each with a one-off cost (4852..4852 and 6735..6735). Neither is marked as rebuilt per row, and neither sits under a dependent subquery.	**Pass**
3. voucher reaches each set by a hash join or by an index lookup on the materialised set	“Single-row covering index lookup on cur using <auto_distinct_key> (d=…, amt=…)”, and the same on old.	**Pass**
4. Each NOT EXISTS is an antijoin or a dependent lookup using an index on Recpt_No, or a hash antijoin	Two “Nested loop antijoin” nodes, each fed by “Index lookup on a/b using Recpt_No (Recpt_No=v.Vnum)”.	**Pass**
5. No ledger table is scanned inside a loop over vouchers	The only table scans on a and b are inside the two union materialisations, which run once. Inside the voucher loop both ledger tables are reached by Recpt_No index lookups.	**Pass**

Stop findings. None present: no full scan of a ledger table under a dependent subquery or inside the voucher loop; no set rebuilt per row or scanned per voucher; the plan reads plainly against points 1–5. **[Observed]**

What the plan predicts. Each ledger table is read twice to build its pair set (about 54,500 rows in all), then each authorised voucher does two index lookups on Recpt_No and two single-row lookups into the materialised sets. The optimizer's row estimates are rough (it expects 798 vouchers after the Aflag filter where R0 counted 7,654), so the cost figures are not a time prediction. The shape is the one Revision 1 asked for; the 36-minute per-voucher passes of the original R20.2 are absent. Runtime remains untested, as Revision 1 said. **[Deduced]**

R20.2r was not run. It needs your separate approval (section 9). If you approve it, it would run with the same gate and capture, with the 120-second hint, after a fresh H1 baseline.

5. Ledger aflag and Bank_Date results (L1, L2)

M1 first (column facts the reads rely on). In both ledger tables aflag is tinyint unsigned, NOT NULL, default 0; Bank_Date is a nullable date; Date is a date; Dr and Cr are decimal(19,2) NOT NULL default 0.00; Recpt_No is int NOT NULL default 0; Discription is varchar(300) NOT NULL default '0'. In voucher, Vnum is int, Amount decimal(12,2), W_Date datetime, Aflag and sup nullable int. In v_inv, V_tkn is bigint, so the joins to Vnum are numeric. **[Observed]**

M2 (engines and indexes). account_runing (11,364 rows) and account_runing_2024 (15,900 rows) are MyISAM with seven non-unique single-column indexes each (ID, index_no, Invoice_Num, L_Code, P_Cap, P_Month, Recpt_No) and **no primary key: ID is not unique**. voucher (7,975 rows) is MyISAM with PRIMARY (ID) and the unique index uq_tpd_voucher_vnum (Vnum). v_inv (8,178 rows) is MyISAM with PRIMARY (ID) only; V_tkn is not indexed. user_acc_tbl is InnoDB (about 78 rows). The MyISAM counts are exact. **[Observed]** The ledger tables' lack of a unique journal id is the structural side of the SX-7 collision risk; it means nothing in the schema stops two journals sharing an ID. **[Deduced]**

L1: how aflag and Bank_Date are spread (11 rows) [Observed]

Table	aflag	Description type	Rows	Dr total	Cr total	Rows with Bank_Date
account_runing	0	hotel reservation accrual	1,353	54,394,139.45	54,608,981.95	0
account_runing	0	lunch or other fee	1,336	7,544,778.02	7,543,778.02	0
account_runing	0	other (text withheld)	4,696	715,633,947.38	715,633,947.38	0

Table	aflag	Description type	Rows	Dr total	Cr total	Rows with Bank_Date
account_runing	3	other (text withheld)	137	76,013,133.65	75,354,147.95	0
account_runing	3	voucher payment	3,842	112,177,908.63	112,173,908.63	0
account_runing_2024	0	brought forward	45	34,616,998.78	34,200,677.67	0
account_runing_2024	0	hotel reservation accrual	2,184	92,513,916.81	92,513,916.81	0
account_runing_2024	0	other (text withheld)	6,823	739,384,055.52	739,384,055.52	0
account_runing_2024	3	brought forward	1	0.00	416,321.11	0
account_runing_2024	3	other (text withheld)	2,343	1,223,802,268.11	1,223,802,268.11	0
account_runing_2024	3	voucher payment	4,504	186,476,391.62	186,476,391.62	0

Reading. Only two aflag values exist, 0 and 3, in both tables; **no row has aflag 1** anywhere, so no tour advance currently stands as “taken out of the advance list” in either table. **Bank_Date is set on none of the 27,264 rows**, which matches the code (nothing writes it). Every “voucher payment” row has aflag 3 and no Bank_Date, the pattern the code predicts. The row totals equal M2′s counts (11,364 and 15,900), so no row was dropped from a category. **[Observed]**

Cross-check with Stage One. The categories that do not balance in account_runing are explained exactly by the eight R21 journals, so the imbalances are known ones, not new: hotel accruals credit-heavy by 214,842.50 = 44,752.50 + 134,257.50 + 35,832.50 (journals 27724, 27725, 28358, single credit legs); lunch or other fee debit-heavy by 1,000.00 (24734: Dr 6,000 against Cr 5,000); aflag 3 other debit-heavy by 658,985.70 = 497,850.38 + 161,135.32 (25915, 25918, single debit legs); aflag 3 voucher payment debit-heavy by 4,000.00 (25804: +5,000; 24654: −1,000). account_runing_2024 balances overall: its brought-forward debits of 34,616,998.78 equal the brought-forward credits 34,200,677.67 + 416,321.11, and every other category balances. **[Deduced from L1 and L2]**

L2: aflag and Bank_Date on the Stage One rows (41 rows, as expected: 11 + 30) [Observed]

The eight unbalanced journals (R21): 24654 (aflag 3: Dr 5,000.00 to 6-9-11-11590, Cr 6,000.00 to 1-6-8-10380, 2026-06-10); 24734 (aflag 0: Dr 6,000.00, Cr 5,000.00, 2026-05-26); 25804 (aflag 3: Dr 10,000.00, Cr 5,000.00, 2026-07-07); 25915 (aflag 3: a single debit of 497,850.38 to 1-6-8-10380, 2026-04-01); 25918 (aflag 3: a single debit of 161,135.32 to 1-6-8-10898, 2026-04-01); 27724, 27725, 28358 (aflag 0: single credits of 44,752.50, 134,257.50 and 35,832.50 to expense accounts, August–September 2026).

The fifteen future-dated journals (R22), 30 rows: 5915 and 5916 (2030-01-01, aflag 3) are a voucher-payment pair for 178,020.00 through 1-6-8-9750 and the bank account 1-6-8-10380; 11851 (2029-03-20, aflag 3) credits 1-6-8-9750 with 4,788,575.00; 25668 and 25669 (2525-04-16, aflag 3) are the first double-looking pair, 25,000.00 each, debiting 1-6-8-10380; 28694 and 28695 (2027-01-04, aflag 0) are the second, 123,310.00 each; 6562 (2037), 11825 and 11827 (2029), 26516–26519 (July 2027) and 28658 (2027-05-01) are aflag 0. Bank_Date is null on all 41 rows.

Reading. On every one of these rows aflag matches the route that would have posted it (3 for voucher-payment-style journals, 0 for the rest), and no row is marked in any way that would take it out of a balance. Nothing here reverses or voids a Stage One finding; each of the 23 journals stands as reported. Which of them should be corrected is the accountant's decision (section 7, Stage C). **[Deduced]**

6. Code reviews F-1 to F-7

The code has four authorisation paths, and none of them refuses a voucher that is already authorised except the payment desk. It also has at least eleven ways the same obligation can be paid or posted twice. **[Observed: code copy, read-only]**

The code copy is identical to the target except TourPaymentDeskController.php, Bulk.php and Vouchers.php, for which the staged target files were used. That rests on R0's comparison and on the 21:07 UTC manifest check, which found the target unchanged since R0. F-5 needs your computer and is still open. File:line references are to that code.

Every authorisation path

Path	Route (POST)	What it does	Guard against a second authorisation
V1	accounts/voucher_auth/update	VoucherPaymentController::update :348. Posts the ledger, then sets Aflag 1. sup, type, amount and payee come from the browser (:351–357). Takes a voucher number before any check (:371).	**None**: it refuses only a voucher marked rej (:375). A second approval posts the ledger again.

Path	Route (POST)	What it does	Guard against a second authorisation
V2/V3	tour/tour_payment_dashboard/paid, /bulk_paid	The desk calls V1 internally **only when Aflag is 0** (target :1457), under the lock `tpd_paid_<voucher>`.	Yes, on the desk path only. The lock does not cover V1 called from other pages.
V4	settings/manual_update/hotel_voucher_accept	Approves every pending voucher with a "Hotel Reservation" line. It posts all of a voucher's lines once per distinct line Invoice (ManualUpdateController :169–359; VoucherPaymentModel grouping by Vnum and Invoice).	None. **The route is live, but its button is commented out** (manual_update.php :52–58), so no screen calls it.
V5	dashboard2/transport/driver_advance/transfer_amount	Creates a new voucher already at Aflag 1 (:564). Posts a ledger pair under the **original** voucher's number (:466–504). Changes the original voucher's amount without checking Aflag (:514).	None. No check that the amount is positive or within balance, or that the drivers differ.

No other current code sets Aflag 1.

Who can reach these routes (F-7)

The order of global filters is sign-in, then AccessPolicy, then UserPermissionFilter (Filters.php :46–48).

- **UserPermissionFilter** passes a request if its path equals, or starts with, a menu address the user holds, or matches that menu's alias list.
  - Group 1 bypasses it.
  - Read-only users (group 34, or profile names with "read-only" or "auditor") are refused on any non-GET request.
  - It has no action-level rights.
- **AccessPolicy** (E8) covers accounts/* only. It is in report mode, so it refuses nothing.
- **CSRF tokens** are sent by the pages (layouts/tf_csrf.php) but checked nowhere: the global csrf filter is commented out (Filters.php :50).

Route	Menu needed to pass the filter	Role check in the controller
delete_advance	dashboard2/transport/driver_advance or a parent	none
hotel_payment (tour/reserve_hotel)	any holder of tour/booking_confirm, through its alias	none
accept_approved_expenses	tour/tour_accounts	approver (r1b)

Route	Menu needed to pass the filter	Role check in the controller
desk initiate, bulk_create, bulk_initiate	tour/tour_payment_dashboard	accountant or batch privilege (Perms)
voucher_auth/update	accounts or accounts/voucher_auth (a user with only Bank Payment Check is refused)	none
hotel_voucher_accept	settings/manual_update	none

What the pages send (F-1)

Route	Page	Confirmation	Double-submit guard	Note
delete_advance	transport_advance.php :769–790	SweetAlert	**none**	shown for every pending row; tour_id comes from the first hidden input on the page, not from the voucher
hotel_payment	booking_details.php :2456–2484	SweetAlert	**none**: Confirm can be pressed again while the first request runs	the page sends extra_val but the controller reads extra (:492)
accept_approved_expenses	approved_expenses_report.php :488–530; payment_approval_v2.php :1454–1487	modal	**none** on the modal's own submit	payable_amount is sent but never used by the server (:2249)
initiate, bulk_create, bulk_initiate	desk pages	form or review screen	buttons disabled; bulk_create also has a one-time token	—
voucher_auth/update	voucher_print.php :379–440; bank_payment_check.php :683–723	SweetAlert, or a bank date only	browser flag or disabled buttons only	voucher_print echoes URL values into hidden inputs **unescaped** (:173–179)

Driver advances (F-2, F-3)

- **Three ways to raise a pending advance** voucher check nothing against each other:
  - tour_transport, on two routes; its duplicate guard is commented out;
  - advanceRequestSubmit (DriverAllocationController :943–976);
  - approval of a “Transport Advance” request (TourAccountsCheckController :695–767).

- Rejecting a Transport Advance request resets **all** of that driver's request rows on the tour (TourAccountsCheckController :770–778), including paid ones, so they can be requested again.
- **The tour marker consignee.transport_advance** holds the pending amount, then 1 once paid. It is a display and completion gate on dashboards. It never blocks a new advance. Rejecting an advance voucher leaves it unchanged.
- **delete_advance** (R1) deletes any pending voucher number, of any type, and resets the marker on whatever tour the browser sends.

Desk libraries and expenses (F-4)

- **Paid** cannot run twice through the desk: the named lock, the finalized guard and the unique lock row prevent it. But the steps are not atomic. A failure after `finalized` is set leaves exchange differences and bank charges unposted, and a retry is refused.
- **Bank Payment Check's mark_finalized** checks neither Aflag nor finalized nor separation. It can finalise an unposted or cancelled voucher, or overwrite a bank date.
- **Batches::prepare** does not exclude **disputed** items, and takes no lock.
- **Batches::byVnum** returns null on any error, so a batch voucher can look ordinary to Bank Payment Check's guard.
- **Separation** refuses only the requester (and the initiator, at Paid). It is not applied to Bank Payment Check or V1.
- **Driver expenses:**
  - approve_expense is routed (Routes.php :275), unused by any page, and **posts the ledger**. accept_approved_expenses would then post the same expense again.
  - The top-up rule at TourAccountsController :2388–2435 is confirmed, and runs for every allocated driver.
  - A cancelled settlement voucher still counts as "already settled".

Other voucher writers (F-6)

- manual vouchers (accounts/voucher_payments/save);
- creditor payments (accounts/creditor_invoice/complete_payment);
- driver settlements, on two identical routes;
- hotel_pay (identical to R2);
- tour_transport (two routes);
- utility bills (update_status and pay, through createUtilityVoucher).

None has a lock, a transaction or a duplicate check. All take numbers from the unlocked counter. NewCreditorInvoiceController::complete_payment123 is not routed; its page posts to an address that returns 404.

Duplicate-payment and double-posting risks

#	Risk	Where	Basis
D1	Re-approving a voucher posts its ledger again (double click in another tab, or Voucher Print plus Bank Payment Check)	V1, VoucherPaymentController :375	Observed code; effect Deduced
D2	Bulk hotel accept posts a voucher with lines on k tours k times	V4	Observed code; no screen calls it today
D3	Accepting expenses for a settled driver raises a top-up equal to all posted expenses, including ones already paid	TourAccountsController :2388–2435	Observed code; effect Deduced
D4	An expense approved through approve_expense, then accepted, is posted twice	DriverExpensesController approve_expense; accept_approved_expenses	Observed code; approve_expense unused by pages
D5	Two simultaneous accepts post every expense twice and raise two vouchers	accept_approved_expenses: no lock, unconditional status change	Deduced
D6	Hotel nights paid by the desk can be paid again through hotel_payment or hotel_pay. The desk sets the flag to 1 (desk :1481) without updating the paid-dollar count, which the old routes compare against	R2, hotel_pay	Observed code; effect Deduced

#	Risk	Where	Basis
D7	Driver settlement charges the transport cost twice and never clears the driver: creation posts Dr 4-8-106-10385 / Cr driver; the voucher line's account is 4-8-106-10385, so approval posts Dr 4-8-106-10385 / Cr bank	DriverPaymentSettlementController :200–310 (and the BookingConfirm copy)	Observed postings; effect Deduced, for the accountant
D8	Two creations, a double submit, or a utility bill accepted and then paid produce two vouchers for one bill	all creation routes	Observed: no duplicate checks
D9	Two writers take the same voucher number. Lines written before a voucher that then fails attach to the other voucher, and are posted with it	InvoiceNumberModel :193–208; Vouchers.php :120	Deduced
D10	An advance transfer moves money without a balance check, posts under the original voucher's number, and leaves the original voucher's amount out of line with its ledger	V5	Observed code
D11	Bank Payment Check finalises without state checks	BankPaymentCheckController mark_finalized	Observed code

Two further findings, not payments.

- `Voucher_No` is written by V1 (:539) but is not in the model's allowed fields, so it is silently dropped.
- voucher_print.php echoes URL values unescaped, which is a cross-site scripting risk.

7. Proposed corrections: affected files, tests and rollback

The corrections come in three stages. The first is a set of code-only guards that stop the double-payment paths without touching data or schema. The second is accounting-rule fixes that need the accountant's view. The third is data corrections that only the accountant can decide. **[Proposal] — nothing in this section is built, tested or installed.**

Stage A. Code-only guards

#	Correction	Closes	Files
A1	**Authorisation guard on V1.** Take the shared lock `avenir:c0:auth`. Re-read the voucher and refuse unless Aflag is 0 and it is not rejected. Use the **stored** sup, type, bank and line amounts instead of the browser's. Refuse if "VOUCHER PAYMENT" rows already exist under the number. Take the voucher number only after the checks. Add the missing Voucher_No allowed field.	D1, C0.15	app/Controllers/Accounts/VoucherPaymentController.php; app/Models/VoucherPaymentModel.php; new app/Libraries/PaymentGuards/AuthGuard.php
A2	**Refuse V4** (hotel_voucher_accept) with a message; keep the route so the refusal is recorded.	D2	app/Controllers/Settings/ManualUpdateController.php
A3	**Refuse delete_advance** (R1). A wrong pending advance is rejected in Bank Payment Check. Remove the Delete button.	R1	DriverAdvancePaymentController.php; Views/pages/dashboard2/transport/transport_advance.php
A4	**Refuse both hotel-advance routes** (R2 and hotel_pay). Hotel advances go through payment requests and the desk. Remove the Pay buttons.	D6, R2	Controllers/Tour/HotelManagementController.php ; Controllers/NewDashboard/hotel/HotelPaymentController.php; Views/pages/tour/booking_details.php

#	Correction	Closes	Files
A5	**Expense acceptance guard.** Lock the tour. Claim each expense with `UPDATE … SET status='posting' WHERE id=? AND status='approved'` before posting it. Raise a top-up only for expenses claimed in this call. Ignore cancelled settlement vouchers. Refuse approve_expense, which is unused and posts the ledger.	D3, D4, D5	Controllers/Tour/TourAccountsController.php; Controllers/NewDashboard/DriverExpensesController.php
A6	**One voucher-number allocator.** incrementVoucherToken takes the desk's lock `tpd_voucher_issue` and checks the number is free. Every creation route writes the voucher **before** its lines and checks each write.	D9	app/Models/InvoiceNumberModel.php; the creation controllers in section 6 (F-6)
A7	**Advance transfer guard.** Require the original voucher to be authorised. Read its amount from the database, not the browser. Require an amount above 0 and not more than the balance, and two different drivers. Use the AUTH lock.	D10	DriverAdvancePaymentController.php
A8	**Finalise guard.** mark_finalized and bulk_finalize refuse unless Aflag is 1, finalized is 0 and the voucher is not cancelled, and apply the separation rule.	D11	app/Controllers/Report/BankPaymentCheckController.php
A9	**Desk fixes.** Narrow the bulk undo to this run's own vouchers and requests. Exclude disputed items in Batches::prepare. Re-read the batch under the lock in R6. Allow a stopped Paid run to finish its exchange and charge postings.	R4, R6	Libraries/TourPayments/Bulk.php, Batches.php; Controllers/Tour/TourPaymentDeskController.php
A10	**CSRF check** on the payment and accounting POST routes, which already send the token, and escape voucher_print.php's hidden inputs.	X4	app/Config/Filters.php; Views/pages/accounts/voucher_print.php

A1–A5 match the containment design in A2 section 9 and Revision 1 section 5. Installing them is the containment you have kept pending.

Stage B. Accounting-rule fixes (need the accountant's view first)

- **B1. Driver settlement posting (D7).** Either the voucher line should debit the driver's account, clearing the accrual, or the settlement should not post at creation. Files: DriverPaymentSettlementController.php and the BookingConfirmController copy.
- **B2. sup = 0 vouchers.** These credit the clearing account and never the bank. The rule for them, and for voucher 7870, is the accountant's call. A1 would refuse sup 0 until that decision is made.
- **B3. Hotel night flags.** V1 approval and rejection update every hotel on those dates (VoucherPaymentController :547, :603), and the paid-dollar count compounds in R2. These matter only if A4 is not chosen.

Stage C. Data corrections (the accountant decides; not code)

- the LKR 298,375.00 difference (section 3);
- the 8 unbalanced journals;
- the 15 future-dated journals;
- the two double-looking pairs;
- the 3,921 vouchers with no authorisation rows under their number.

No correcting journal is proposed by me.

Meaningful tests

Tests run on an isolated copy under a separate approval, never on development or live. Each test checks the database rows, not only the screen.

Test	Expected result
T-A1a	Approve a voucher twice, in two tabs: the second is refused; one set of ledger rows; one voucher number used
T-A1b	Approve with sup, amount or bank altered in the request: refused, or the stored values are used; the ledger matches the stored lines
T-A1c	Reject a sup = 2 voucher: no clearing journal posted
T-A1d	Desk Paid and Bank Payment Check approve at once: one posts, the other is refused

Test	Expected result
T-A2/A3/A4	POST to each refused route: refusal with a message; no row in any table changes; an audit row is written
T-A5a	Accept the same tour's expenses twice at once: each expense posted once; one voucher
T-A5b	Settled driver plus one new expense: the top-up equals the new expense only
T-A6	Two vouchers created at once on different screens: two distinct numbers; each line under its own voucher
T-A7	Transfer more than the balance, zero, or to the same driver: refused
T-A8	Finalise a pending, cancelled or already-finalised voucher: refused; bank date unchanged
T-A9	Force a failure mid-bulk: only this run's rows are removed; another session's request is untouched
T-A10	Each payment page still works with the token; a POST without the token is refused
Regression	The full screen battery and the existing PHPUnit suites, before and after; counts of every payment table unchanged by read-only screens

Rollback

- **Code only:** stage A changes no schema and no data.
- **Kit:** each install ships with a copy of every file it replaces and a checksum list taken before and after.
- **To roll back:** put the before-copies back and confirm their checksums match the pre-install list. Filters.php alone undoes A10.
- **Backup:** a full verified backup of files and database is taken before any install, as in earlier stages, so data can be restored if a defect wrote anything.
- **Triggers:** roll back if any T-A test fails on the copy, or if after install a payment screen errors or a ledger count changes unexpectedly.

Installed changes: none.

8. Evidence folder, checksums and missing evidence

Where. By your correction of 15 October, the folder is inside the project's writable folder rather than at C:\AvenirEvidence: **C:\wamp64\www\avenir\writable\AvenirEvidence\2026-10-14-stage1**. It is under www, but it is not web-served: writable.htaccess holds "Require all denied" and the project's root .htaccess rewrites every request to public. No application file was touched. **[Observed]**

What it holds (114 files, every one listed in SHA256SUMS):

Folder	Files	Content
stage1-results\	10	the nine Stage One result files R20.1–R23.2 and their README (free text in R22.1/R22.2 summarised, as before)
statements\stage1\	12	T1, T2, T3 and R20.1–R23.2 as sent in Stage One
statements\revision1\	14	the twelve texts run here, plus G1 (reference, not run) and R20.2r (not run), each matching its Revision 1 SHA-256
code-checks\	10	f3, f4, f7, f9, f11_before, f11_after, r30_vs_r0, f5_tree_check, review_files_copy.sha256, r0_final_files.sha256
followup-run\	68	for each of the 21 steps: the raw response (tokens removed), the rows file and the metadata file; the E1 plan; README; RUN_LOG; the sender (runner.js) and the saver (avr_save.py)
reports\	0	empty; see missing evidence
SHA256SUMS	1	114 entries, computed on your computer and verified there: 0 mismatches. Its own SHA-256 is 97ac666d49badf3a391018590d1b2607daa36bbd69213e4e07abaa4919df72bc

Integrity chain. Each raw response's SHA-256 was computed in the browser before transfer, verified in the session workspace after transfer, and matches the value in SHA256SUMS computed on your computer (for example 01_T1_response.json: 81e3b2a3… at all three points; 20_E1_plan.txt: 320b8c96…). Every session token in the raw responses was replaced by "[token removed]" (5 to 11 occurrences per file); a scan of the saved files for any remaining 32-hex token value found none. No password or API key appears in any file: the statements read none, and the sign-in was yours. **[Observed]**

Missing or partial evidence [Observed]:

- **The old R20.2 response: only partly recoverable; its result was not recovered.** The browser pane's network log still held the request (id 14036.8129) at 11:20 UTC on 15 October, but only the first part of its body can be read: phpMyAdmin's page-settings block, which carries a session token. No result table and no message are readable, and the query was not rerun. Nothing of it has been saved to the folder, because Revision 1 made saving it conditional on your approval and on redaction of the token. If you want the readable part kept, say so; otherwise it stays out.
- **Stage One raw responses.** For T1–T3 and R20.1–R23.2 only the parsed result files exist; the raw phpMyAdmin responses were not kept at the time. Their row counts and phpMyAdmin timings are in the Stage One report, not in files.
- **R30 code-check outputs F0, F1, F2, F6, F8 and F10** were read in the session and reported in R0; they were not saved as files, so code-checks\ holds only the outputs that existed as files (listed above).
- **PDF exports** of the Stage One report, the follow-up package and Revision 1 are not yet in reports. They can be exported after you have read this report, so that the exported copies are the final ones.
- **Not evidence about live.** Everything here was read on localhost. The live staff system, avenir_rv and the cloud test database were not accessed.

9. Decisions needed to proceed to implementation

These are the decisions that would let a build start. Nothing has been built or installed. **[Proposal]**

1. **R20.2r.** Run it or not, given E1's plan (section 4). It needs your separate approval.
2. **Stage A scope.** Approve all of A1–A10, or name the subset and its order. My recommended order is A1, A2, A3, A4, A5, A8, then A6, A7, A9, A10.
3. **R1 and R2.** Refuse, as recommended, or guard.
4. **sup = 0 under A1.** Whether A1 refuses sup = 0 vouchers, including voucher 7870, until the accountant sets the rule (B2). The alternative is to let them through as today.
5. **The accountant's review.** When Ravi may be contacted and which records he may receive: B1 (settlement posting), B2 (sup = 0), and the stage C data items. Until you say so, nothing is sent.
6. **Test environment.** Stage A must be tested on an isolated copy before any install. This investigation excluded the cloud test database and avenir_rv, so you would need to name the copy to use or approve building a new one.
7. **Install target and window.** Development (your localhost) only, after a full verified backup, at a time you choose. Live is not covered by any of this.

8. **CSRF and E8 (A10).** Whether to turn on token checking for payment routes, and whether E8 stays in report mode. Pages already send the token, but any page that does not would start failing.
9. **Staff changes.**
    - Transport staff would ask Accounts to reject a wrong advance instead of deleting it.
    - Reservations staff would raise hotel advances as payment requests.
    - Accounts would see refusals on double approvals.
    - Who tells them, and when, is your decision.
10. **Acceptance and rollback.** Sign-off on the T-A tests in section 7 as the acceptance criteria, and on the rollback triggers.

A0, D14 and containment implementation remain pending. Stage A1–A5 is the containment in a refined form. Approving it would settle the containment decision, but not A0 or D14.

Added 15 October, after the run (nothing above changed):

- **E1 condition 1.** Whether the two MySQL Notes (#1276, section 4) count as "warnings" for the rule. On MySQL's own levels they are Notes, and the plan meets conditions 2–5; if you accept that reading, R20.2r is runnable under the existing text and checksum, with your separate approval.
- **Voucher 7134, the bank side.** Whether to approve one further read-only statement on the bank ledger (1-6-8-10380 and the clearing of 1-6-8-9750) to see whether the payment itself was posted once or twice. The text would be prepared for your checksum approval first; nothing is run until then.
- **The old R20.2 response.** Whether the readable first part (phpMyAdmin's settings form, token removed) should be saved to the evidence folder or left out. Its result is not recoverable either way.
- **PDF exports.** Whether to export the Stage One report, the follow-up package, Revision 1 and this report into reports\ now, which would complete the folder.

10. Draft bank-side read for voucher 7134 (B1) — prepared, NOT RUN

Status. Drafted on 15 October at your request. It has not been sent anywhere, not to localhost, not to any copy. It runs only after you approve this exact text by its checksum, and under the same gate, capture and stop rules as T1–E1. **[Proposal]**

Question it answers. For voucher 7134 (amount 298,375.00, finalized with bank date 2026-07-13 per R23.1), G3 showed two voucher-payment journals crediting 1-6-8-9750, and G2r showed no journal under the number debiting 1-6-8-9750. B1 looks for the bank side: every ledger row filed under the voucher number (7134) or the voucher's record id (7145), and

every row on the payable 1-6-8-9750 or the Commercial Bank account 1-6-8-10380 whose debit or credit equals 298,375.00 or 596,750.00, in both ledger tables. Rows that match both ways appear once.

Scope. Schema avenir only: account_runing, account_runing_2024 and user_acc_tbl (ids only). Read-only; capped at 60 seconds by hint. No free text is output: descriptions are reduced to a category, the Invoice_Num column is shown only as '0' or '(withheld)', and no bank account number, name, password column or guest detail is read. The indexes on Recpt_No and L_Code (M2) serve both halves of the WHERE.

Expected. At least the four G3 rows of journals 26061 and 26075. Any further row shows where, and how many times, the bank side of 7134 was posted. No row is dropped; nothing is corrected.

Assumption to confirm at run time. Columns Dt, Cashier and Invoice_Num are confirmed on account_runing (G3 read them); on account_runing_2024 they are taken to exist because the 6 October dump gives both tables the same structure, but M1 did not list them. If MySQL reports an unknown column, that is a stop, and the text is revised for a new checksum.

Statement text (SHA-256 74e1b6fab6305ffbcf9cab8ecc3ef2d4a1d3cb2e66e0cd48d69c2b3b5341544f, over the exact text below, no trailing newline):

```
SELECT /*+ MAX_EXECUTION_TIME(60000) */ 'account_runing' AS ledger_table,
a.ID AS journal_id, a.Recpt_No AS receipt_no, a.L_Code, a.Dr, a.Cr, a.Date,
a.Dt AS entered_at,
       CASE WHEN UPPER(a.Discription) LIKE 'VOUCHER PAYMENT%' THEN 'voucher
payment'
            WHEN UPPER(a.Discription) LIKE 'BANK%' THEN 'bank (text
withheld)'
            WHEN a.Discription IS NULL OR a.Discription = '' THEN '(empty)'
            ELSE 'other (text withheld)' END AS description_type,
       CASE WHEN a.Invoice_Num = '0' THEN '0' ELSE '(withheld)' END AS
line_invoice_marker, a.aflag,
       (SELECT GROUP_CONCAT(u.id ORDER BY u.id) FROM avenir.user_acc_tbl u
WHERE u.user = a.Cashier) AS entering_login_ids
FROM avenir.account_runing a
WHERE a.Recpt_No IN (7134, 7145)
   OR (a.L_Code IN ('1-6-8-9750', '1-6-8-10380') AND (a.Dr IN (298375.00,
596750.00) OR a.Cr IN (298375.00, 596750.00)))
UNION ALL
SELECT 'account_runing_2024', b.ID, b.Recpt_No, b.L_Code, b.Dr, b.Cr, b.Date,
b.Dt,
       CASE WHEN UPPER(b.Discription) LIKE 'VOUCHER PAYMENT%' THEN 'voucher
payment'
```

```
            WHEN UPPER(b.Discription) LIKE 'BANK%' THEN 'bank (text
withheld)'
            WHEN b.Discription IS NULL OR b.Discription = '' THEN '(empty)'
            ELSE 'other (text withheld)' END,
       CASE WHEN b.Invoice_Num = '0' THEN '0' ELSE '(withheld)' END, b.aflag,
       (SELECT GROUP_CONCAT(u.id ORDER BY u.id) FROM avenir.user_acc_tbl u
WHERE u.user = b.Cashier)
FROM avenir.account_runing_2024 b
WHERE b.Recpt_No IN (7134, 7145)
   OR (b.L_Code IN ('1-6-8-9750', '1-6-8-10380') AND (b.Dr IN (298375.00,
596750.00) OR b.Cr IN (298375.00, 596750.00)))
ORDER BY 1, 7, 2, 5 DESC, 4;
```

How it would run, if approved. As step B1 after a fresh H1 baseline, then H1 again (expected: set +1, failed and exceeded unchanged); one statement, SELECT only, checksum gate before sending; raw response saved with tokens removed, rows file and metadata, under the evidence path and by the capture route that the structural compatibility register leaves for your decision (register SC-06, U-7). Stop on any error, a row count differing from phpMyAdmin's total, or a counter moving unexpectedly.

Approval wording, if you want it run: "B1 approved: SHA-256 74e1b6fa…, localhost avenir only, read-only, one run." Until then it stays a draft.